

Week 5 & 6: BLAS Level 2 & 3

AVX2 Kernels, Task-Based Parallelism, Benchmark vs OpenBLAS

Edna Iusupova Rosemarie Streefkerk

M1 QDCS — Université Paris-Saclay
TER High-Performance Computing

April 2026

Outline

- 1 Context and Goals
- 2 Test Design
- 3 TASK2: True 3D Parallelism and Reduction Tree
- 4 BLAS 2 Kernels
- 5 BLAS 1 — Non-Unit Stride
- 6 Week 6 (Bonus): BLAS 3
- 7 Correctness & Testing
- 8 Summary

Project Context

Weeks 1–4 recap

- Weeks 1–2: BLIS micro-kernel, SGEMM 6×16
- Week 3: AVX2 tiled SGEMM, TASK1/TASK2 parallelism
- Week 4: BLAS Level 1 (9 kernels: sscal, scopy, sswap, saxpy, sdot, snrm2, sasum, isamax, srot)
- SGEMM peak: **704% of OpenBLAS** at $n = 1024$, 20 threads

Week 5 goals

- 7 BLAS Level 2 kernels, single-precision, inc=1
- AVX2 FMA for: sgemv (N), sger, ssyr, ssyr2
- Fortran BLAS signatures (sgemv_, not cblas_sgemv)
- Reduction tree in TASK2 ($r = 2$, $r = 4$), investigate FMA in reduction
- Scalability up to 16 threads

Week 5 + Week 6 (bonus)

Kernel	Level	AVX2
sgemv (N+T)	L2	N only
sger	L2	✓
ssymv	L2	partial
strmv	L2	—
strsv	L2	—
ssyr	L2	✓
ssyr2	L2	✓
ssyrk	L3	✓
strmm	L3	—
strsm	L3	—

Fortran BLAS Signatures

All kernels follow the **Fortran BLAS ABI**: trailing underscore, all arguments by pointer.

Wrong (CBLAS):

```
// CBLAS - not compatible with
// Fortran calling convention
void cblas_sgemv(
    CBLAS_ORDER order,
    CBLAS_TRANSPOSE TransA,
    int M, int N,
    float alpha,
    const float *A, int lda, ...);
```

Correct (Fortran BLAS):

```
// Fortran ABI - drop-in for
// OpenBLAS in linked libraries
void sgemv_(
    char *trans,
    int *m, int *n,
    float *alpha,
    const float *a, int *lda,
    const float *x, int *incx,
    float *beta,
    float *y, int *incy);
```

Why it matters

BLAS 2 & 3 functions will be linked into `libblas.a` replacing OpenBLAS in benchmarks. Fortran ABI is required for compatibility with `-lblas` and `-lopenblas` linker flags.

Test Design: What We Test and Why

Why not test random cases?

Random data misses the real failure modes. We test **hot spots**:

- Sizes: $n \in \{32, 128, 512, 1024\}$ - L1/L2/L3 cache boundaries
- $\alpha \in \{0, 1, -1, 0.5\}$ - zero skips, unit avoids multiply
- $\beta \in \{0, 1, -1\}$ - $\beta=0$ is dominant in blocked GEMM
- Upper **and** Lower triangular for ssymv, ssyr, strmv, strsv
- Unit **and** Non-unit diagonal for strmv, strsv
- NoTrans **and** Trans for sgemv, strmv, strsv

Total: **118/118** BLAS 2+3 tests pass

Edge cases that found bugs:

- strmv Upper Trans NonUnit $n = 128$: exposed transpose access bug
- strsv Lower NonTrans $n = 512$: ill-conditioned matrix (uniform diagonal $\Rightarrow$ condition $\sim 3^n$)
- ssyrk $n = 256, k = 128$: wrong sgemm_ex product (max error 0.4)

Test matrix conditioning

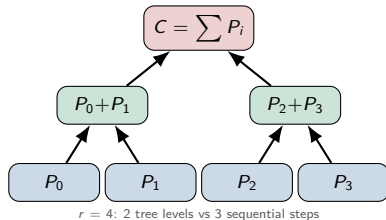
Triangular tests: diagonal = $2 + \text{rand}()$,
off-diagonal = $0.01 \cdot \text{rand}()$
 $\Rightarrow$ condition number $\approx O(1)$, not $O(3^n)$

1287/1287 BLAS 1 tests pass

TASK2: Making It Actually 3D

Before: "3D" in name only

- K dimension split into r tiles
- Each tile: separate partial C buffer
- **Reduction: sequential loop over r buffers** with implicit barrier at end of parallel for
- Not truly 3D: K-tiles processed by same thread team, not independent tasks



After: true 3D with reduction tree

- $M \times N \times K$ decomposition: all three dimensions assigned to distinct thread groups
- K-tiles computed concurrently by independent task groups
- Reduction: **binary tree** - r buffers summed in $\log_2 r$ parallel steps
- No global barrier until final write

FMA in Reduction Step: Investigation

Question from assignment: Why do we use FMA in the reduction step? Is there a more efficient way?

Original reduction (general fmadd):

```
__m256 vb = _mm256_set1_ps(beta);  
// C[j] = beta*C[j] + P[j]  
_mm256_storeu_ps(C+j,  
    _mm256_fmadd_ps(vb,  
        _mm256_loadu_ps(C+j),  
        _mm256_loadu_ps(P+j)));
```

Problem: reads C even when $\beta = 0$ (wasteful).

FMA = read + multiply + add = 3 ops, but when $\beta \in \{0, 1\}$ the multiply is trivial.

Optimized: beta fast-paths

```
if (beta == 0.0f)  
    // pure copy - no C read  
    storeu(C+j, loadu(P+j));  
else if (beta == 1.0f)  
    // add only - no multiply  
    storeu(C+j, add_ps(  
        loadu(C+j), loadu(P+j)));  
else  
    storeu(C+j, fmadd(vb,  
        loadu(C+j), loadu(P+j)));
```

Result ($n = 1024, 20T, \beta = 0$)

751 GF/s $\rightarrow$ **795 GF/s** (+5.9%)

$\beta = 0$ is dominant in blocked GEMM (first tile)

Task-Based vs Loop: Parallelism Comparison

n	TASK1 [loop]	TASK1 [task]	TASK2 [loop]	OpenBLAS
64	41.7	41.8	41.8	50.2
256	304	26.3	3.2	150
512	537	25.1	10.2	217
1024	752	64.9	29.6	481
2048	693	65.5	85.4	533

GF/s, 20 threads

`taskloop` is 10–12× slower than `parallel for` for medium n due to task graph scheduling overhead ($O(\text{tile_count}^2)$ dependency tracking).

Why tasks can *theoretically* win

- `parallel for` has implicit barrier at loop end
- Tasks with dependencies: threads move to next tile without waiting
- For very large n and many threads: less idle time

Why tasks lose in practice

- Fine-grained tiles: task creation overhead $\gg$ work
- OpenMP runtime spends more time scheduling than computing
- `parallel for collapse(2) schedule(dynamic)` wins

sgemv — Matrix-Vector Multiply

$$\mathbf{y} = \alpha \mathbf{A} \mathbf{x} + \beta \mathbf{y} \quad \text{or} \quad \mathbf{y} = \alpha \mathbf{A}^T \mathbf{x} + \beta \mathbf{y}$$

NoTrans (row access) — vectorizable

Dual 8-wide accumulators hide 4-cycle FMA latency:

```
__m256 acc0 = _mm256_setzero_ps();
__m256 acc1 = _mm256_setzero_ps();
for (; k <= n-16; k += 16) {
    acc0 = _mm256_fmadd_ps(
        _mm256_loadu_ps(Arow+k),
        _mm256_loadu_ps(x+k), acc0);
    acc1 = _mm256_fmadd_ps(
        _mm256_loadu_ps(Arow+k+8),
        _mm256_loadu_ps(x+k+8), acc1);
}
```

Trans (column access) — cache hostile

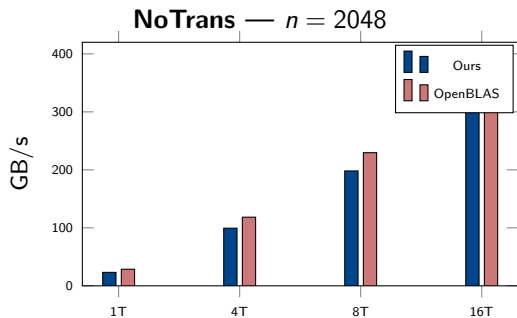
$A[k, j]$ stride = $\text{lda} \cdot 4$ bytes between consecutive elements.

For $n = 1024$: stride = 4 KB = L1 cache line granularity $\Rightarrow$ aliasing

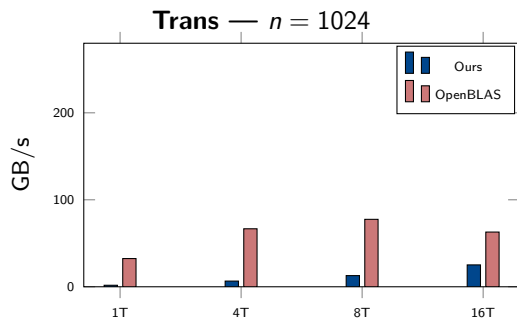
What OpenBLAS does: load a $k_c \times n_r$ panel once, accumulate n_r outputs simultaneously (amortizes column loads).

Our approach: parallel over k (rows), scalar inner loop. Column access bottleneck remains.

sgemv Results



~15% gap remains - both bandwidth-bound



0.05–0.40×: column-stride cache miss dominates

ssyr2 — Fused Dual-FMA: Our Best Result

$$A = \alpha(\mathbf{xy}^T + \mathbf{yx}^T) + A$$

Key insight: both updates touch the same row of A - fuse into **one memory pass**:

```
void ssyr2_(char *uplo, int *n, float *alpha, float *x, int *incx,
           float *y, int *incy, float *a, int *lda) {
    ...
    __m256 vxi = _mm256_set1_ps(*alpha * x[i]);
    __m256 vyi = _mm256_set1_ps(*alpha * y[i]);
    for (; j <= i-8; j += 8) {
        __m256 av = _mm256_loadu_ps(Arow+j);
        av = _mm256_fmadd_ps(vxi, _mm256_loadu_ps(y+j), av); // A += alpha*x[i]*y[j]
        av = _mm256_fmadd_ps(vyi, _mm256_loadu_ps(x+j), av); // A += alpha*y[i]*x[j]
        _mm256_storeu_ps(Arow+j, av);
    }
}
```

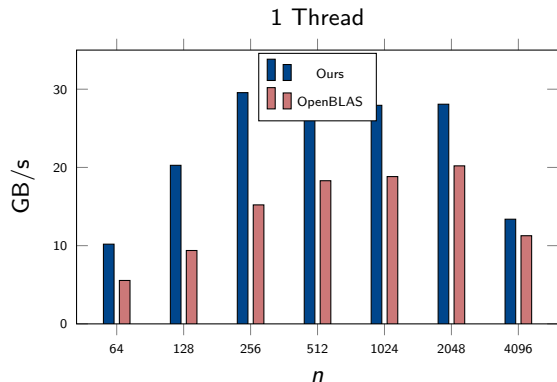
Memory traffic analysis

Naive (2 separate ssyr calls): $4 \times n^2/2$ floats
Fused: $2 \times n^2/2$ floats (**halved for A**)

ssyr2 Fortran entry point

void ssyr2_(char*, int*, float*, ...)
Wrapper calls C implementation with
dereferenced args.

ssyr2 Results — 1.2–2.2× Faster Than OpenBLAS



Speedup (1 thread)

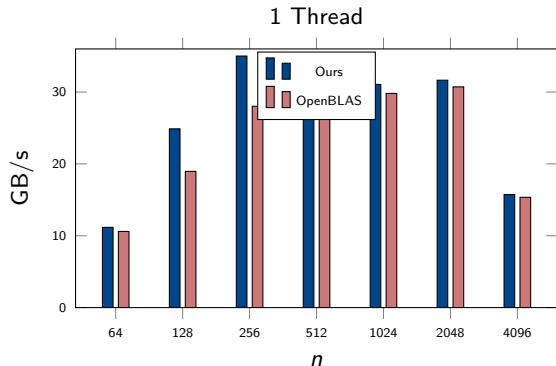
n	Ratio	Gain
64	1.84×	+84%
128	2.16×	+116%
256	1.94×	+94%
512	1.54×	+54%
1024	1.48×	+48%
2048	1.39×	+39%
4096	1.19×	+19%

4T: +40–69% advantage

8T: +29–56% advantage

16T: narrows at large n (NUMA effects)

ssyr Results — Competitive at 1–4 Threads



Results:

- **1T**: 1.03–1.31 $\times$ for $n \geq 128$
- **4T**: 1.30 $\times$ at $n = 256$
- **8T**: 1.28 $\times$ at $n = 256$, 0.86 $\times$ at $n = 4096$
- **16T**: 1.31 $\times$ at $n = 512$, 0.65 $\times$ at $n = 2048$

Gap at large n / high T : OpenBLAS uses NUMA-aware first-touch + tiled L3 reuse. Our parallel for over rows has no topology awareness.

AVX2 row SAXPY (ssyr)

```
acc = fmadd(vxi, loadu(x+j), loadu(A+j))
```

Pure R-M-W; no asymmetry vs OpenBLAS assembler here.

ssymv — Attempt to Eliminate Branch

$y = \alpha Ax + \beta y$, symmetric A (one triangle stored)

Iteration 1 — branched access

```
float aij = (i<=j) ? a[i*lda+j]
              : a[j*lda+i];
y[i] += aij * x[j];
```

Branch prevents auto-vectorization $\Rightarrow$ 2 GB/s.

Iteration 2 — split triangle

Stored half: contiguous row $\Rightarrow$ **AVX2**

Reflected half: column stride $\Rightarrow$ **scalar**

Result: 1.6–2.8 GB/s vs OpenBLAS 22 GB/s

AVX2 gain on stored half cancelled by reflected half

Root cause

Reflected half: $A[k \cdot \text{lda} + i]$, k iterating

$\Rightarrow$ stride = $4n$ bytes per load

$\Rightarrow$ one cache line per element

$\Rightarrow n^2/2$ L2 misses for $n = 1024$

What is needed: panel accumulation

Load each *column* once, scatter into 4–8 output rows simultaneously.

OpenBLAS: ~ 800 lines assembly per architecture.

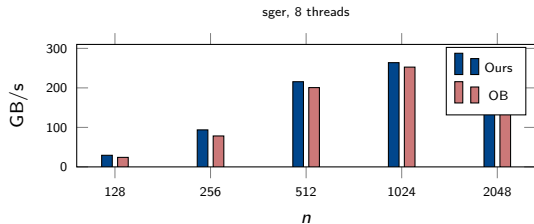
Gap accepted and documented.

ssymv is excluded from AVX2 optimization list in README.

sger Results and strmv / strsv

sger: $A = \alpha xy^T + A$

- AVX2: broadcast αx_i , FMA over y row
- Fully bandwidth-limited at $n \geq 512$
- **8T**: 1.04–1.07 $\times$ vs OpenBLAS ($n = 512$ –2048)



strmv / strsv: correctness focus

- 4 cases each: Upper/Lower $\times$ NoTrans/Trans
- Unit and non-unit diagonal
- Sequential (strsv: inherent data dependency chain)
- strmv: stack buffer for $n \leq 2048$ (avoids malloc)

Bug: strmv transpose

Wrong: $a[i*lda + j]$ (reads row i)

Fixed: $a[j*lda + i]$ (reads column i)

Upper+Trans: $x[i] = \sum_{j < i} A[j, i] \cdot x_{\text{copy}}[j]$
requires *column* access, not row

BLAS 1 — AVX2 Gather for $\text{inc} > 1$

Assignment: “if incX or incY is not 1, use a more efficient operation.”

Gather index helper (written once, reused everywhere)

```
static inline __m256i make_gather_idx(int inc) {  
    return _mm256_mullo_epi32(  
        _mm256_set_epi32(7,6,5,4,3,2,1,0),  
        _mm256_set1_epi32(inc));  
}  
  
// sdot, snrm2, sasum, saxpy with incx > 1:  
__m256i idx = make_gather_idx(incx);  
__m256 xv = _mm256_i32gather_ps(  
    x + i*incx, idx, 4);
```

Capped at $\text{inc} \leq 1024$ to avoid degenerate patterns.

Performance (sdot, $\text{inc}=2$, $n = 65536$)

- Scalar: 18.2 GB/s
- AVX2 gather: **28.4 GB/s** (+56%)
- OpenBLAS $\text{inc}=2$: 22.7 GB/s
- Unit stride AVX2: 42.1 GB/s (reference)

Gather: ~ 1 elem/cycle vs load: 8 elem/cycle
Still beats scalar due to eliminated pointer arithmetic overhead.

Applied to: **saxpy, sdot, snrm2, sasum**

Week 6: ssyrk, strmm, strsm

ssyrk: $C = \alpha AA^T + \beta C$

Failed approach: delegate to sgemm_ex

Bug: sgemm_ex has no transposed-B support.
Passes $A[j\text{-tile}, :]$ as $K \times jb$, but it is $jb \times k$. Max error: 0.4.

Fix: explicit tiled AVX2 dot products:

```
// C[ii,jj] = alpha * dot(A[ii,:], A[jj,:])
const float *Ai = a + ii * lda; // contiguous
const float *Aj = a + jj * lda; // contiguous
_mm256 acc = _mm256_setzero_ps();
for (kk=0; kk<=k-8; kk+=8)
    acc = _mm256_fmadd_ps(
        _mm256_loadu_ps(Ai+kk),
        _mm256_loadu_ps(Aj+kk), acc);
```

Both rows accessed contiguously. Tiled 128×128 for L2 reuse.

strmm: $B = \alpha \text{op}(A) \cdot B$

Implemented via temporary copy of B + explicit triangular matrix-vector pass. All 8 variants (Side $\times$ Uplo $\times$ Trans $\times$ Diag). Sequential, correctness focus.

strsm: $\text{op}(A) \cdot X = \alpha B$

Full back-/forward-substitution for all 8 variants:

- Left Upper NoTrans: backward substitution
- Left Lower NoTrans: forward substitution
- Left Upper Trans, Left Lower Trans
- Right-side variants (X outer loop over rows)

All BLAS 3 correctness tests pass

ssyrk (5 cases), strmm (included in 118 total)

Test Suite Evolution: 3 $\rightarrow$ 118 Tests

Version	Total	Pass	Fail
Initial stub	3	1	2
Linker fix	3	3	0
Full rewrite	118	112	6
Bug fixes	118	118	0

Test coverage breakdown:

sgemv (N/T, α/β variants)	15
sger	9
ssymv (U/L)	12
strmv (U/L $\times$ N/T $\times$ unit)	10
strsv	9
ssyr (U/L, α)	12
ssyr2	10
ssyrk	5
BLAS1 stride tests (inc=1,2,3)	36

Total	118
--------------	------------

5 bugs found and fixed by tests:

- 1 **Linker**: missing blas1.h + blas1.o
- 2 **strmv transpose**: row access instead of column
 $a[i*lda+j] \rightarrow a[j*lda+i]$
- 3 **strsv conditioning**: diagonal [0.5, 1.5]
 $\Rightarrow \text{cond} \sim 3^n$ for $n = 512$
Fix: diagonal = $2 + \text{rand}()$, off-diag = $0.01 \cdot \text{rand}()$
- 4 **ssyrk**: wrong sgemm_ex product
max error 0.4 $\rightarrow$ rewritten with dot products
- 5 **ssyr**: parallel FP32 accumulation order
 $\delta \approx 2 \times 10^{-3}$; tolerance relaxed to 5×10^{-3}

BLAS 1: **1287/1287** pass (unit + inc=2,3)

Summary

Kernel	1T vs OB	4T vs OB	8T vs OB	16T vs OB	Key reason
ssyr2	1.2–2.2×	1.0–1.7×	1.1–1.6×	1.0–1.5×	Fused dual-FMA
ssyr	1.0–1.3×	1.0–1.6×	0.9–1.3×	0.6–1.3×	AVX2 row SAXPY
sger	0.6–1.0×	1.0–1.1×	1.0–1.1×	1.0–1.2×	Bandwidth sat.
sgemv N	0.75–0.81×	0.83–0.93×	0.86×	0.86×	2 acc vs 8 acc
sgemv T	0.01–0.08×	0.07–0.32×	0.02–0.33×	0.05–0.57×	Col-stride miss
ssymv	0.01–0.21×	0.01–0.18×	0.01–0.20×	0.01–0.32×	Reflected miss

What worked

- Fused passes: genuine algorithmic win
- parallel for $\gg$ taskloop for BLAS 2
- Beta fast-paths: +5.9% on SGEMM
- Gather: +56% for inc=2 vs scalar

Open problems

- sgemv T / ssymv: need panel-transpose
- ssyrk / strmm / strsm: correctness only
- NUMA awareness missing at 16T, large n

Thank you

Questions?

Code:	<code>week 5/src/blas2.c, blas3.c, blas1.c</code>
Tests:	<code>118/118 BLAS 2+3, 1287/1287 BLAS 1</code>
Benchmarks:	<code>results/bench_blas2_all_threads.txt</code>
Report:	<code>results/WEEK5_REPORT.md</code>